

Sensor GPS

El GPS es un sensor de posicionamiento. Funciona reconociendo marcas de la cinta que esta en la cancha y calculando coordenadas (x, y) y orientación (heading).

- Mide posición absoluta en milímetros (x, y).
- Mide orientación en grados (0° – 360°).
- Necesita calibración al inicio (rotación en círculo).
- Ventaja: evita la acumulación de error de encoders y giroscopios.
- Limitación: sensible a mala calibración o interferencias

Control Proporcional (KP)

El código implementa un **control proporcional (P)** para movimientos y giros.

3.1 Explicación

- **Error** = diferencia entre valor deseado y valor medido.
- **Salida** = $KP \times \text{Error}$.

3.2 Constantes

- **KP_TURN** = **0.8** → sensibilidad en giros.
- **KP_DRIVE** = **0.5** → velocidad proporcional a la distancia.
- **KP_HEADING** = **1.0** → corrección de orientación mientras avanza.

3.3 Ejemplo de cálculo

Si el robot necesita girar 90° y está en 45° :

- $\text{Error} = 90 - 45 = 45^{\circ}$.
- $\text{Velocidad} = 0.8 \times 45 = 36\%$.
- Motores giran en sentidos opuestos con velocidad 36%.

4. Funciones Principales del Código

4.1 angleDifference()

- Normaliza la diferencia angular entre dos headings.
- Mantiene el valor en el rango $[-180^\circ, 180^\circ]$ para evitar saltos de 360° .

4.2 turnToHeading(double targetHeading)

- Objetivo: girar al heading deseado.
- Flujo:
 - Se obtiene el heading actual (GPS.heading).
 - Se calcula error con angleDifference().
 - Mientras el error sea mayor al umbral (TURN_THRESHOLD):
 - Se calcula velocidad proporcional ($KP_TURN \times \text{error}$).
 - Motores izquierdos y derechos giran en sentidos contrarios.
 - Se aplica **timeout** de 10s como seguridad.

4.3 driveToPosition(double targetX, double targetY)

- Objetivo: avanzar hacia un punto absoluto (x, y).
- Flujo:
 - Se leen posiciones actuales (GPS.xPosition, GPS.yPosition).
 - Se calcula la distancia al objetivo con Pitágoras:

$$d = \sqrt{(x_{target} - x_{actual})^2 + (y_{target} - y_{actual})^2}$$

- Se calcula el heading deseado con atan2.
- Se determina error de heading actual vs deseado.
- Se calcula velocidad de avance proporcional a la distancia.
- Se corrigen velocidades de cada lado según el error angular.
- Se repite hasta llegar dentro del umbral (DRIVE_THRESHOLD).

4.4 pre_auton()

- Calibra GPS.
- Espera hasta que termine.
- Resetea encoders.

4.5 autonomous()

- Ejemplo: cuadrado de 500 mm $\times$ 500 mm.
- Secuencia:
 - Avanzar a (500, 0).
 - Girar a 90°.
 - Avanzar a (500, 500).
 - Girar a 180°.
 - Avanzar a (0, 500).
 - Girar a 270°.
 - Volver a (0, 0).

6. Ejemplo de Recorrido en cancha

Al ejecutar autonomous() en un cuadrado:

- **Inicio:** (0,0), heading 0°.
- **Paso 1:** avanza a (500,0).
- **Paso 2:** gira a 90°.
- **Paso 3:** avanza a (500,500).
- **Paso 4:** gira a 180°.
- **Paso 5:** avanza a (0,500).
- **Paso 6:** gira a 270°.
- **Paso 7:** vuelve a (0,0).

Cada vez el GPS corrige la trayectoria absoluta.

